МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра Н2 Программная инженерия и интеллектуальные системы
шифр наименование кафедры, по которой выполняется работа

Дисциплина **Языки и средства разработки интернет-приложений**
наименование дисциплины

ПРАКТИЧЕСКАЯ РАБОТА

№4
номер задания (при наличии)

Аналитика, визуализация и генерация документов

Тема: «Склад небольшого магазина»

при наличии указать тему учебно-практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы ЗО721

_____ Бахтина Н.А.
подпись фамилия и инициалы

_____
дата сдачи

ПРОВЕРИЛ

_____ Мажайцев Е.В.
подпись фамилия и инициалы

Оценка / балльная оценка _____

_____
дата проверки

г. Санкт-Петербург
2026 г.

Цель работы: научиться агрегировать данные средствами базы данных, представлять их в виде диаграмм и показателей, а также формировать документы установленных форматов по данным приложения.

Задание

Часть 1. Аналитический экран

1. Отдельный экран аналитики, доступ к которому определяется ролями из ТЗ.

2. Не менее трех числовых показателей и не менее трех диаграмм разных типов. Тип выбирается по задаче: динамика во времени – линейная, сравнение категорий – столбчатая, доля в целом – круговая или кольцевая, и только при числе категорий не более пяти-шести.

3. Агрегирование выполняется в базе данных (GROUP BY, агрегатные функции). Сервер возвращает клиенту готовые сводные данные, а не все записи.

4. Фильтр по периоду и еще одному параметру из варианта меняет все диаграммы и показатели одновременно.

Часть 2. Генерация документов

1. Не менее двух документов разных форматов. Один из них указан в вашем варианте, второй выбирается их docx, xlsx, pdf.

2. Документ формируется сервером по данным из базы с учетом выбранных фильтров и скачивается по кнопке в интерфейсе.

3. В документе есть заголовок, дата формирования, таблица данных и итоговая строка или итоговые показатели.

4. Документ docx формируется по шаблону, который хранится в репозитории, а не собирается в коде абзац за абзацем.

5. Кириллица отображается корректно во всех форматах. Для pdf используемый шрифт встраивается в документ.

Выполнение работы

1. Разработка аналитического экрана и серверной агрегации

Для того, чтобы построить графики, базе данных нужны были данные о продажах. Был написан скрипт «seed_sales.py», который сгенерировал 20 случайных продаж за последние полгода. На рисунке 1 показан успешный результат его выполнения.

```
(venv) Mac:sklad-backend macally$ python seed_sales.py
/Users/macally/Desktop/sklad-backend/seed_sales.py:22: DeprecationWarning: datetime.datetime.u
tcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware object
s to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  sale_date = datetime.datetime.utcnow() - datetime.timedelta(days=days_ago)
База данных успешно наполнена тестовыми продажами!
(venv) Mac:sklad-backend macally$
```

Рисунок 1 – Наполнение БД тестовыми продажами

Согласно заданию, серверная часть должна выполнять агрегацию данных (GROUP BY, агрегатные функции) и возвращать клиенту готовые сводные показатели. В файле «main.py» были реализованы маршруты для получения статистики. На рисунках 2-3 представлен код маршрутов аналитики с использованием «func.sum» и «group_by» (фрагмент кода «main.py»).

```python
# --- АНАЛИТИКА (ПР 4) ---

# 1. Сводные показатели (цифры для дашборда)
@app.get("/api/v1/analytics/summary")
def get_summary(month: str = None, db: Session = Depends(get_db)):
    query = db.query(func.sum(models.Sale.total_amount))
    # Если месяц выбран, фильтруем продажи по этому месяцу
    if month and month != "all":
        query = query.filter(func.to_char(models.Sale.date, 'YYYY-MM') == month)

    total_sales = query.scalar() or 0
    total_products = db.query(func.count(models.Product.id)).scalar() or 0
    low_stock_count = db.query(func.count(models.Product.id)).filter(models.Product.current_stock <= models.Product.min_stock).scalar() or 0

    return {
        "total_sales": round(total_sales, 2),
        "total_products": total_products,
        "low_stock_count": low_stock_count
    }

# 2. Оборот по месяцам (для линейного графика)
@app.get("/api/v1/analytics/sales_by_month")
def get_sales_by_month(month: str = None, db: Session = Depends(get_db)):
    query = db.query(
        func.to_char(models.Sale.date, 'YYYY-MM').label('month'),
        func.sum(models.Sale.total_amount).label('total')
    )
    if month and month != "all":
        query = query.filter(func.to_char(models.Sale.date, 'YYYY-MM') == month)

    sales = query.group_by('month').order_by('month').all()
    return [{"month": s.month, "total": round(s.total, 2)} for s in sales]
```

Рисунок 2 – Фрагмент кода main.py (функции «get_summary» и «get_sales_by_month»

```python
# 3. Остатки по категориям (для столбчатого графика)
@app.get("/api/v1/analytics/stock_by_category")
def get_stock_by_category(month: str = None, db: Session = Depends(get_db)):
    # Если месяц выбран, показываем ПРОДАЖИ по категориям за этот месяц
    if month and month != "all":
        query = db.query(
            models.Category.name,
            func.sum(models.SaleItem.quantity).label('total_stock')
        ).join(models.Product, models.Category.id == models.Product.category_id)\
         .join(models.SaleItem, models.Product.id == models.SaleItem.product_id)\
         .join(models.Sale, models.SaleItem.sale_id == models.Sale.id)\
         .filter(func.to_char(models.Sale.date, 'YYYY-MM') == month)\
         .group_by(models.Category.name)

    # Если выбрано "Все месяцы", показываем ТЕКУЩИЕ ОСТАТКИ
    else:
        query = db.query(
            models.Category.name,
            func.sum(models.Product.current_stock).label('total_stock')
        ).join(models.Product).group_by(models.Category.name)

    stock = query.all()
    return [{"category": s.name, "total_stock": s.total_stock} for s in stock]
```

Рисунок 3 – Серверная агрегация данных (main.py)

Проверка работы API была выполнена через «Swagger UI». На рисунке 4 представлен общий список всех маршрутов, включая CRUD-операции, аналитику и генерацию документов.



Рисунок 4 – Маршруты API для аналитики и документов (Swagger UI)

Проверка числовых показателей была выполнена через Swagger с применением фильтра по конкретному месяцу (2026-10). На рисунке 5 представлен ответ сервера со сводными числовыми показателями (общей выручкой, количеством товаров и числом товаров с дефицитом) за выбранный период.

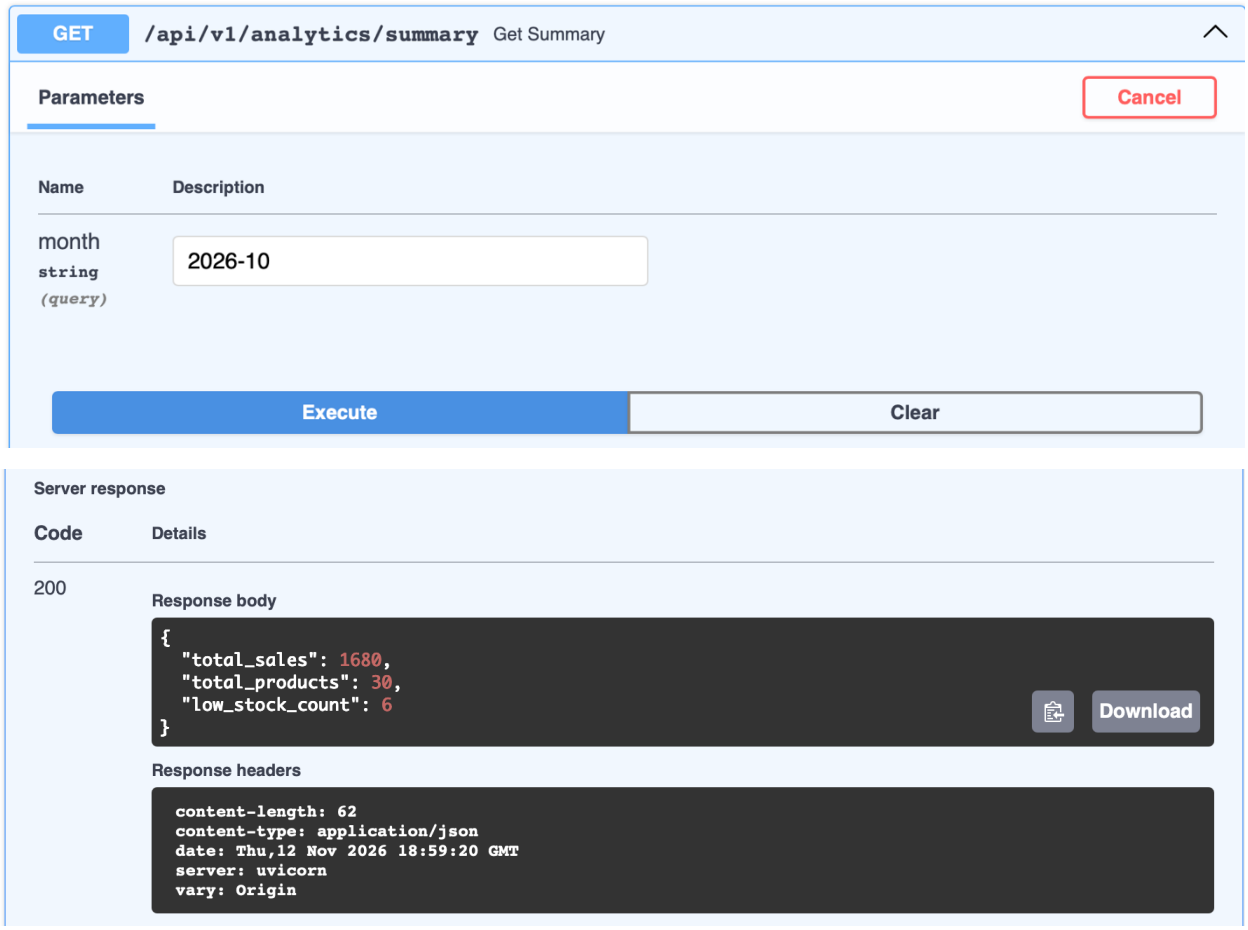


Рисунок 5 – Сводные числовые показатели (summary)

Из рисунка 5 видно, что сервер выполнил запрос (код 200) и вернул три готовых числовых показателя:

- total_sales (общая сумма продаж);

- total_products (всего товаров);

- low_stock_count (товаров с низким остатком).

Для построения графиков сервер отдает готовые агрегированные данные. На рисунке 6 представлен ответ сервера с оборотом по месяцам с применением фильтра (выбран конкретный месяц «2026-07») для линейного графика, а на рисунке 7 – данные по остаткам в разрезе категорий (для столбчатого и кольцевого графиков).

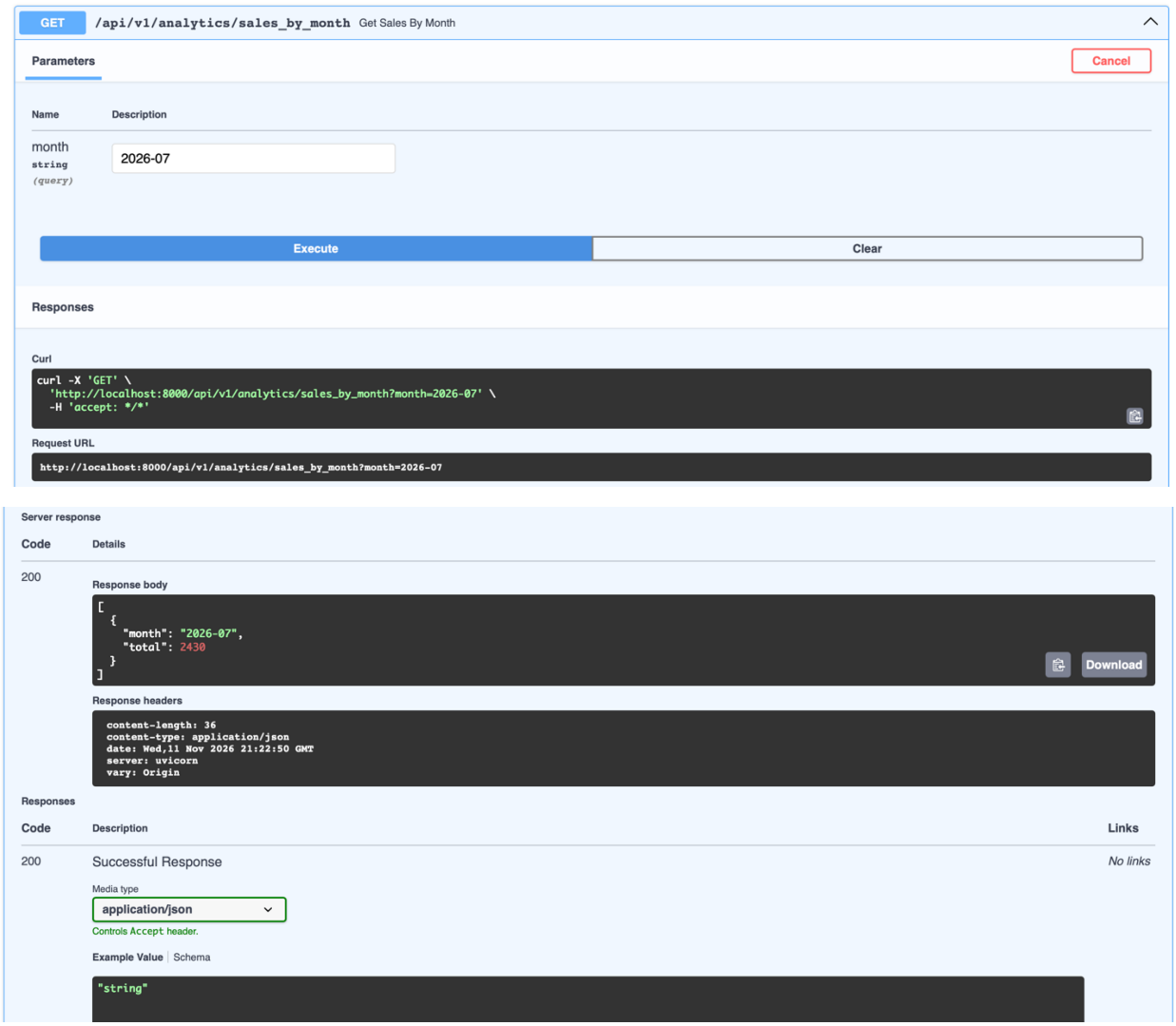


Рисунок 6 – Данные оборота по месяцам (sales_by_month)

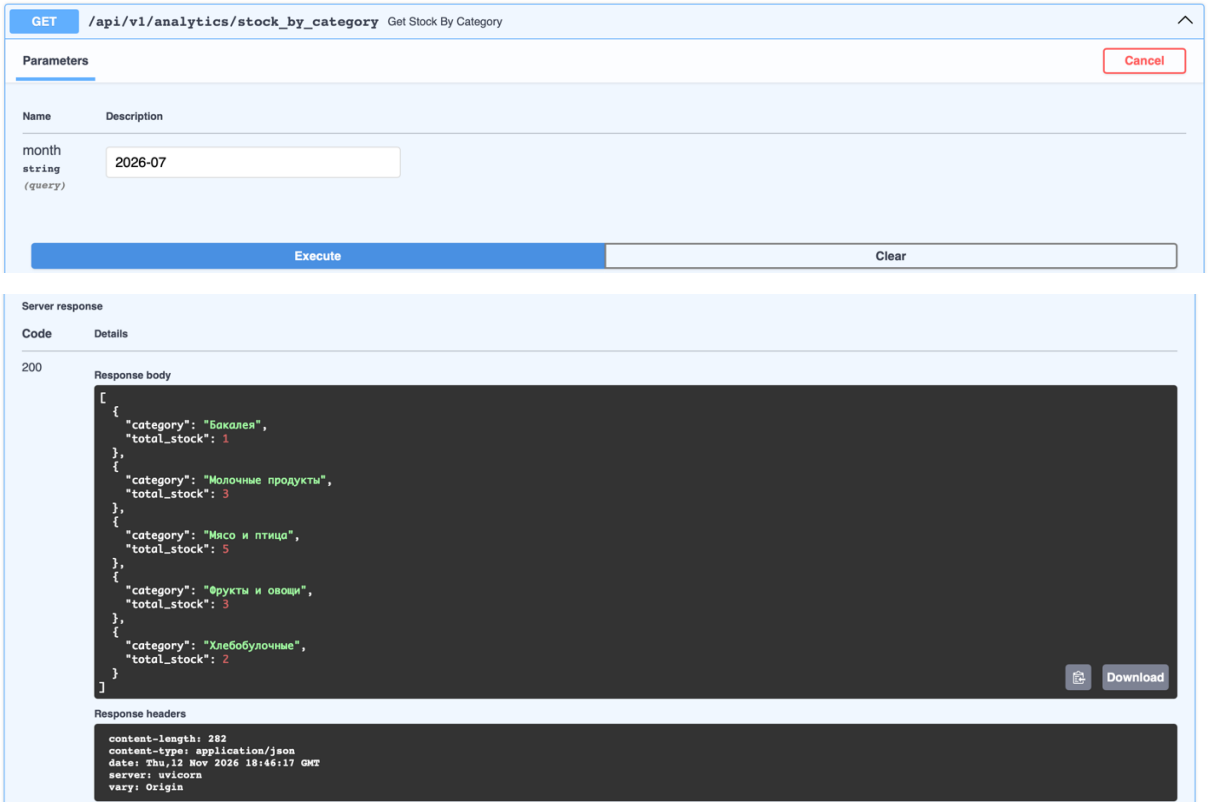


Рисунок 7 – Данные остатков по категориям (stock_by_category)

2. Визуализация данных на клиенте

Для отображения графиков на клиентской части (Vue.js) была установлена и настроена библиотека «Chart.js». На рисунке 8 представлен код экрана аналитики.

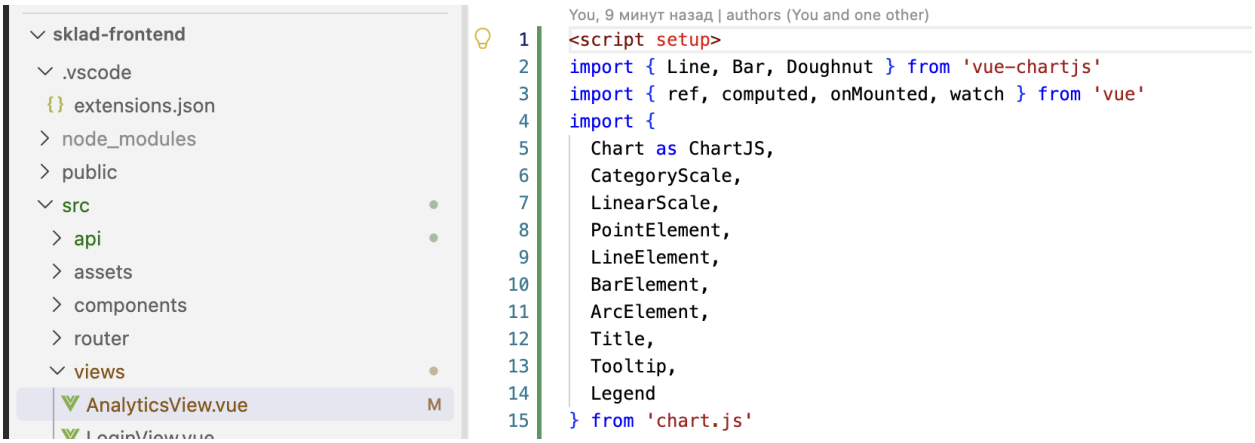


Рисунок 8 – Код экрана аналитики (Analytics.vue)

На рисунках 9-10 представлен итоговый экран дашборда. Реализованы три числовых показателя (карточки) и три диаграммы разных типов: линейная (динамика оборота), столбчатая (сравнение остатков по категориям) и кольцевая (доля категорий в общем остатке).

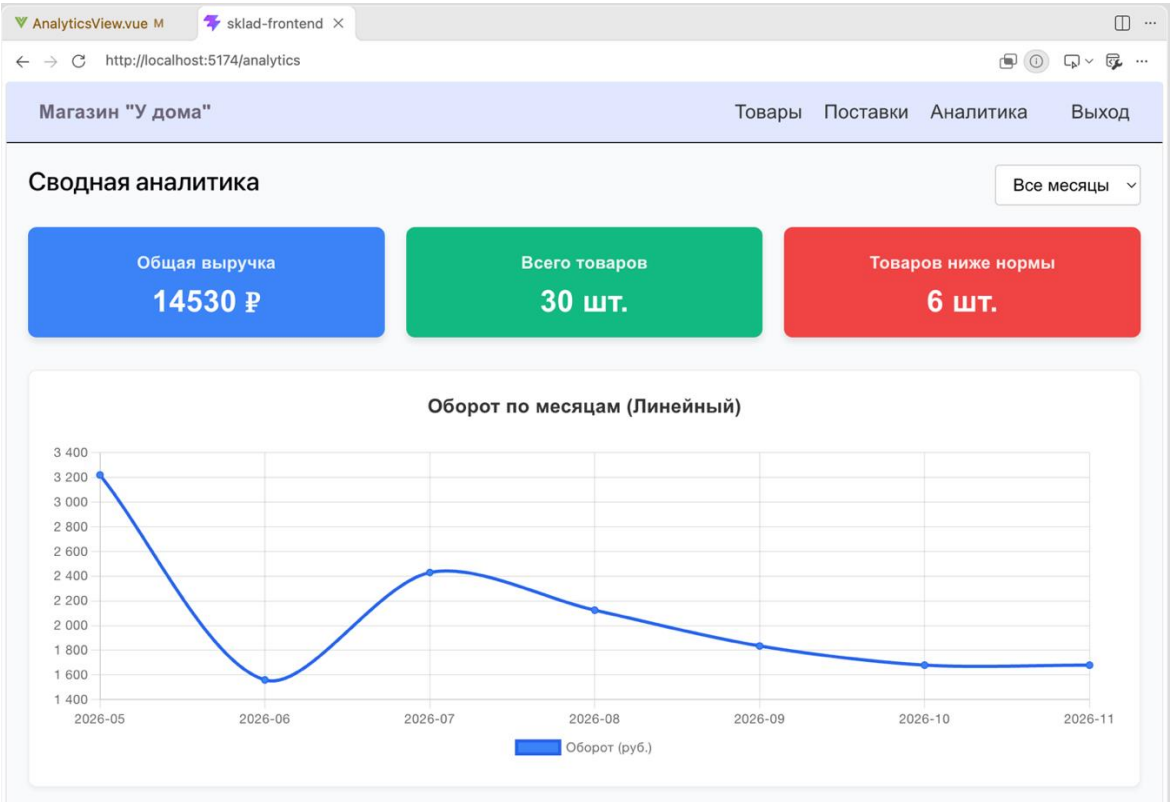


Рисунок 9 – Экран сводной аналитики с линейным графиком

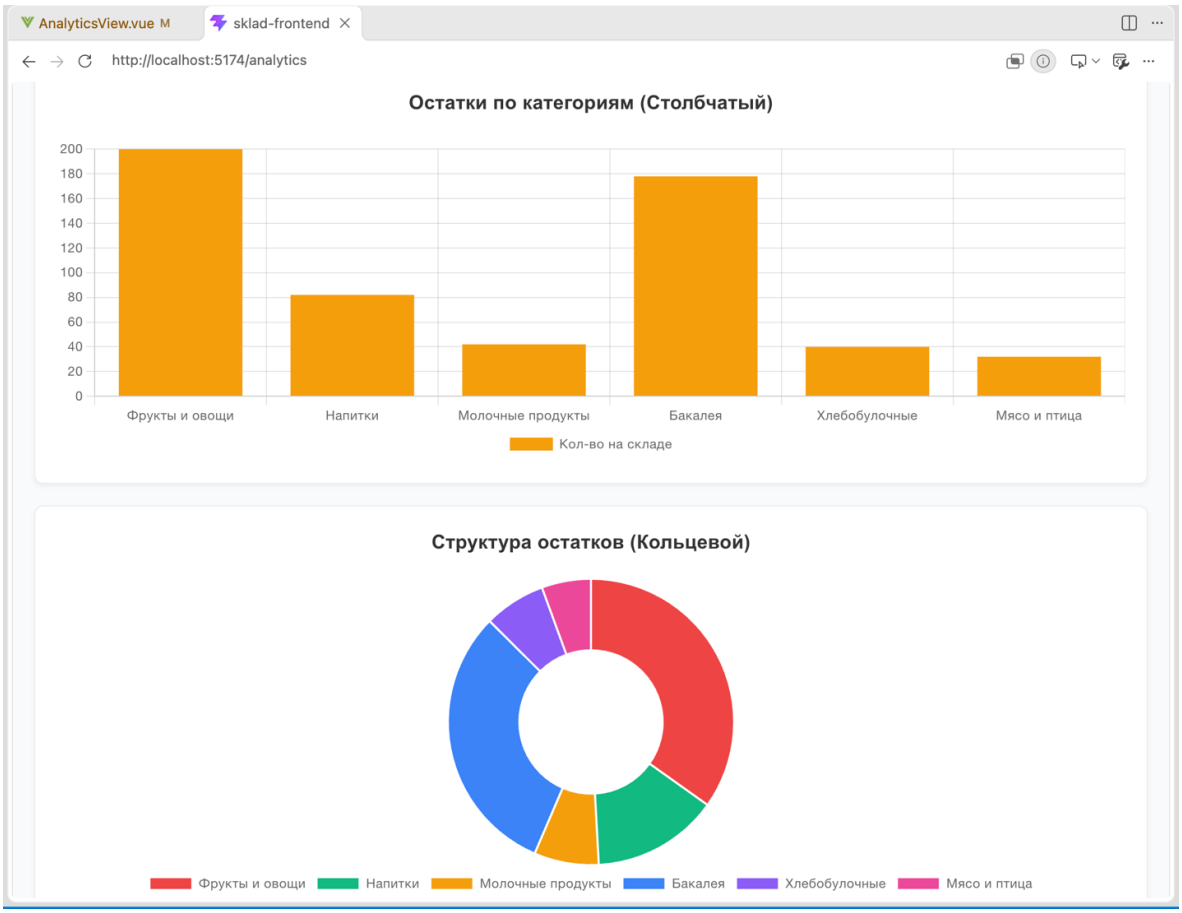


Рисунок 10 – Экран сводной аналитики со столбчатой и кольцевой диаграммами (все месяцы)

На рисунке 9 видна динамика продаж по месяцам на линейном графике. На рисунке 10 столбчатая диаграмма сравнивает категории товаров между собой (больше всего на диаграмме показано «фруктов и овощей»), а кольцевая – показывает структуру склада в процентах.

Для проверки работы фильтра по периоду был выбран конкретный месяц. На рисунке 11 – 12 показано, что при выборе месяца «2026-06» общая выручка пересчитывается, линейный график отображает данные только за выбранный период, а столбчатая и кольцевая диаграммы перестраиваются, отображая количество проданных товаров по категориям за этот месяц. Это подтверждает выполнение требования о фильтрации всех показателей и диаграмм одновременно.

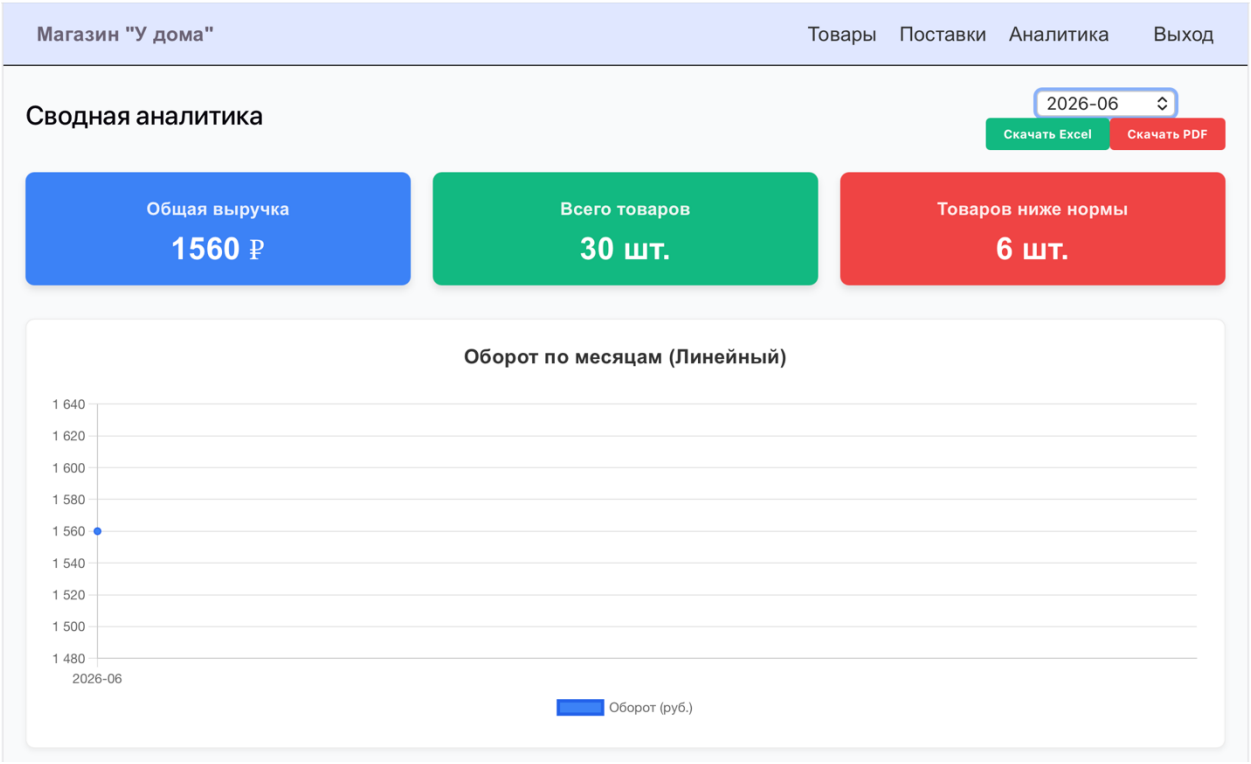


Рисунок 11 – Экраны сводной аналитики склада магазина и линейный график выручки

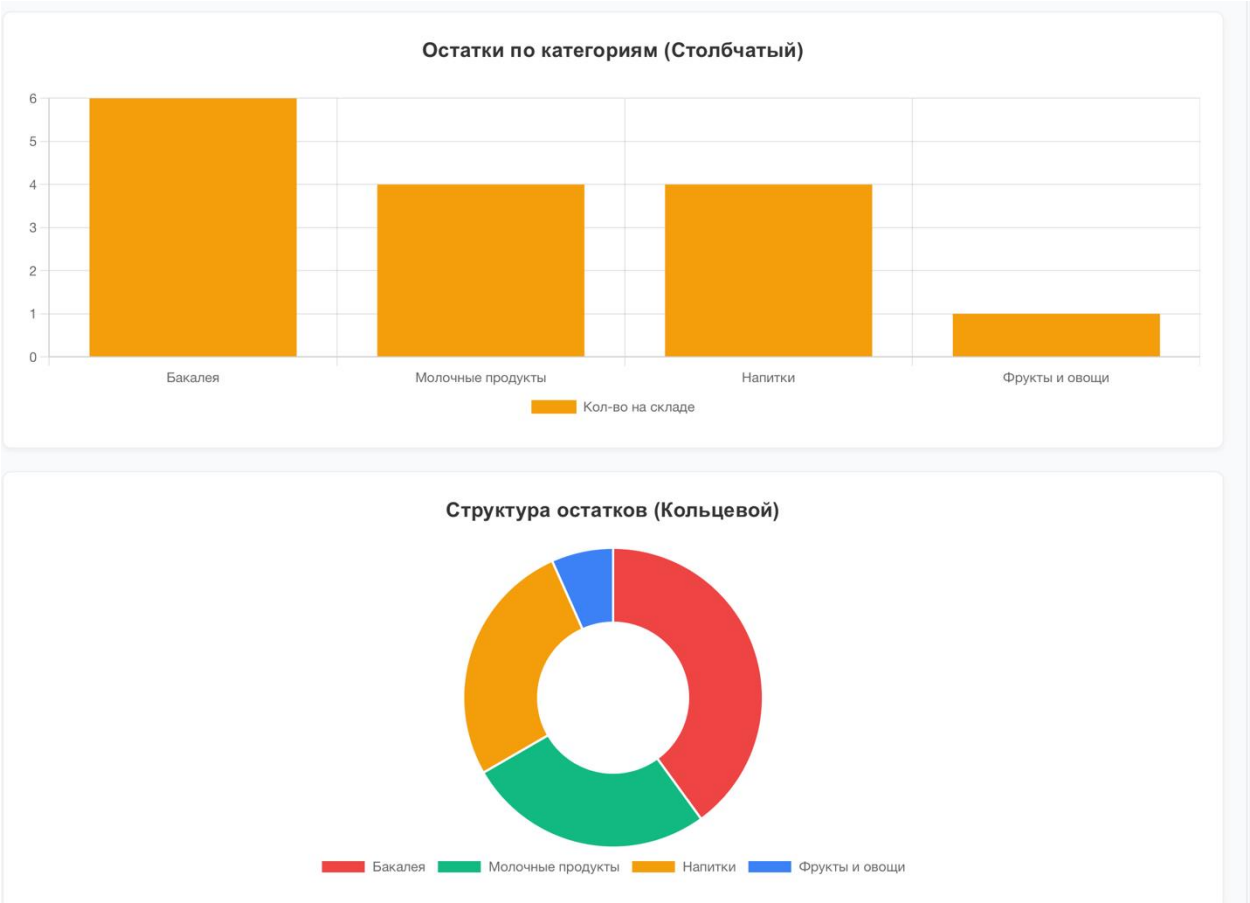


Рисунок 12 - Экран сводной аналитики со столбчатой и кольцевой диаграммами (за определенный период, 2026-06)

3. Генерация документов

В соответствии с требованием ТЗ, сервер формирует документы по данным из базы. Были созданы два маршрута:

1) для генерации PDF-накладной (reportlab);

2) для генерации Excel-ведомости (openpyxl).

На рисунке 13-14 представлен код генерации документов.

```python
# --- ГЕНЕРАЦИЯ ДОКУМЕНТОВ (ПР 4) ---

# 1. Генерация Товарной накладной (PDF)
@app.get("/api/v1/documents/invoice/{sale_id}")
def generate_invoice_pdf(sale_id: int, db: Session = Depends(get_db)):
    sale = db.query(models.Sale).filter(models.Sale.id == sale_id).first()
    if not sale:
        raise HTTPException(status_code=404, detail="Продажа не найдена")

    items = db.query(models.SaleItem).filter(models.SaleItem.sale_id == sale_id).all()

    buffer = io.BytesIO()
    p = canvas.Canvas(buffer, pagesize=A4)

    try:
        from reportlab.pdfbase import pdfmetrics
        from reportlab.pdfbase.ttfonts import TTFont
        pdfmetrics.registerFont(TTFont('Arial', '/System/Library/Fonts/Supplemental/Arial.ttf'))
        pdfmetrics.registerFont(TTFont('Arial-Bold', '/System/Library/Fonts/Supplemental/Arial Bold.ttf'))
        font_name = 'Arial'
        bold_font_name = 'Arial-Bold'
    except:
        font_name = 'Helvetica'
        bold_font_name = 'Helvetica-Bold'

    # Заголовок
    p.setFont(font_name, 18)
    p.drawString(220, 800, "Товарная накладная")

    p.setFont(font_name, 12)
    p.drawString(50, 760, f"Номер чека: {sale.id}")
    p.drawString(50, 740, f"Дата: {sale.date.strftime('%d.%m.%Y')}")

    # Подготовка данных для таблицы
    table_data = [["Наименование", "Кол-во", "Цена", "Сумма"]]

    for item in items:
        prod = db.query(models.Product).filter(models.Product.id == item.product_id).first()
        prod_name = prod.name if prod else "Неизвестно"
        table_data.append([
            prod_name,
            str(item.quantity),
            f"{item.selling_price:.2f}",
            f"{item.quantity * item.selling_price:.2f}"
        ])

    # Итоговая строка
    table_data.append(["", "", "ИТОГО:", f"{sale.total_amount:.2f}"])

    # Создаем таблицу с шириной колонок
    table = Table(table_data, colWidths=[250, 60, 80, 100])

    # Настраиваем стиль таблицы (рамки, цвета, выравнивание)
    style = TableStyle([
        ('BACKGROUND', (0, 0), (-1, 0), colors.lightgrey), # Фон заголовка
        ('TEXTCOLOR', (0, 0), (-1, 0), colors.black),
        ('ALIGN', (0, 0), (-1, -1), 'CENTER'), # Выравнивание по центру
        ('ALIGN', (0, 1), (0, -1), 'LEFT'),    # Названия товаров по левому краю
        ('FONTNAME', (0, 0), (-1, -1), font_name),
        ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
        ('BACKGROUND', (0, 1), (-1, -2), colors.white), # Белый фон для данных
        ('BACKGROUND', (0, -1), (-1, -1), colors.lightgrey), # Фон для ИТОГО
        ('GRID', (0, 0), (-1, -1), 1, colors.black), # Черные рамки
        ('FONTNAME', (0, -1), (-1, -1), bold_font_name) # Жирный шрифт для ИТОГО
    ])
    table.setStyle(style)

    # Рисуем таблицу на холсте (x=50, y=650)
    w, h = table.wrapOn(p, 500, 500)
    table.drawOn(p, 50, 650 - h)

    p.showPage()
    p.save()
    buffer.seek(0)

    return StreamingResponse(buffer, media_type="application/pdf", headers={"Content-Disposition": f"attachment; filename=invoice_{sale.id}.pdf"})
```

Рисунок 13 – Реализация генерации PDF - файла (фрагмент кода «main.py»)

```python
# 2. Генерация Ведомости остатков (Excel)
@app.get("/api/v1/documents/stock_excel")
def generate_stock_excel(db: Session = Depends(get_db)):
    products = db.query(models.Product).all()

    wb = openpyxl.Workbook()
    ws = wb.active
    ws.title = "Остатки на складе"

    # Заголовки
    headers = ["ID", "Название", "Цена", "Остаток", "Мин. остаток"]
    ws.append(headers)

    # Стили для заголовков
    for cell in ws[1]:
        cell.font = Font(bold=True)
        cell.alignment = Alignment(horizontal="center")

    # Данные
    for prod in products:
        ws.append([prod.id, prod.name, prod.price, prod.current_stock, prod.min_stock])

    # Автоширина колонок
    for col in ws.columns:
        max_length = max(len(str(cell.value)) for cell in col if cell.value)
        ws.column_dimensions[col[0].column_letter].width = max_length + 2

    buffer = io.BytesIO()
    wb.save(buffer)
    buffer.seek(0)

    return StreamingResponse(buffer, media_type="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",

                             headers={"Content-Disposition": "attachment; filename=stock_report.xlsx"})
```

Рисунок 14 – Реализация генерации Excel - файла (фрагмент кода «main.py»)

Для скачивания документов на экране аналитики были добавлены соответствующие кнопки в интерфейсе (рисунок 15).

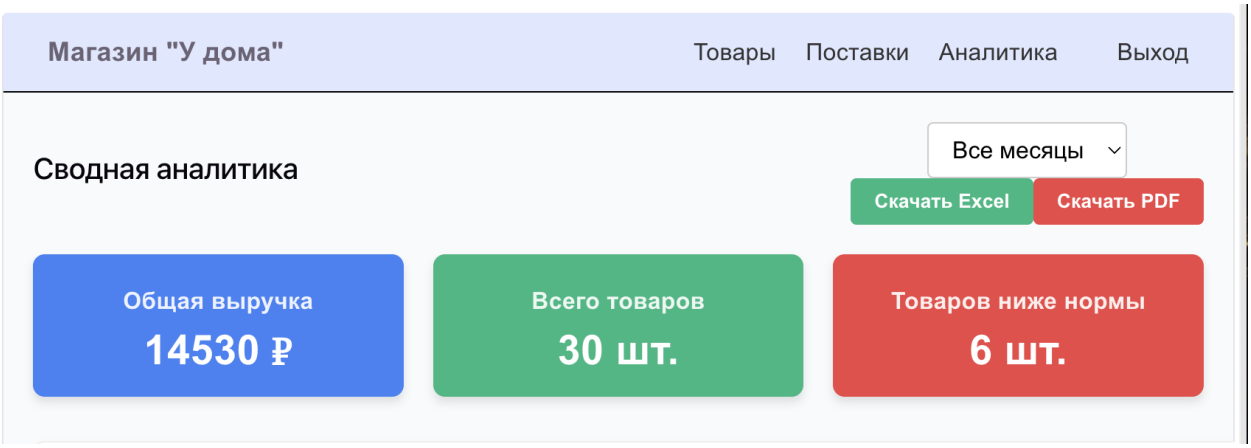


Рисунок 15 – Экран аналитики с кнопками скачивания документов

В результате работы сервер корректно формирует документы. Так как товарная накладная формируется на конкретную транзакцию (продажу), в запросе указывается параметр «sale_id» (номер чека), а не фильтр по месяцу. На рисунке 16 представлен ответ сервера с готовой ссылкой на скачивание PDF – накладной для чека № 1.

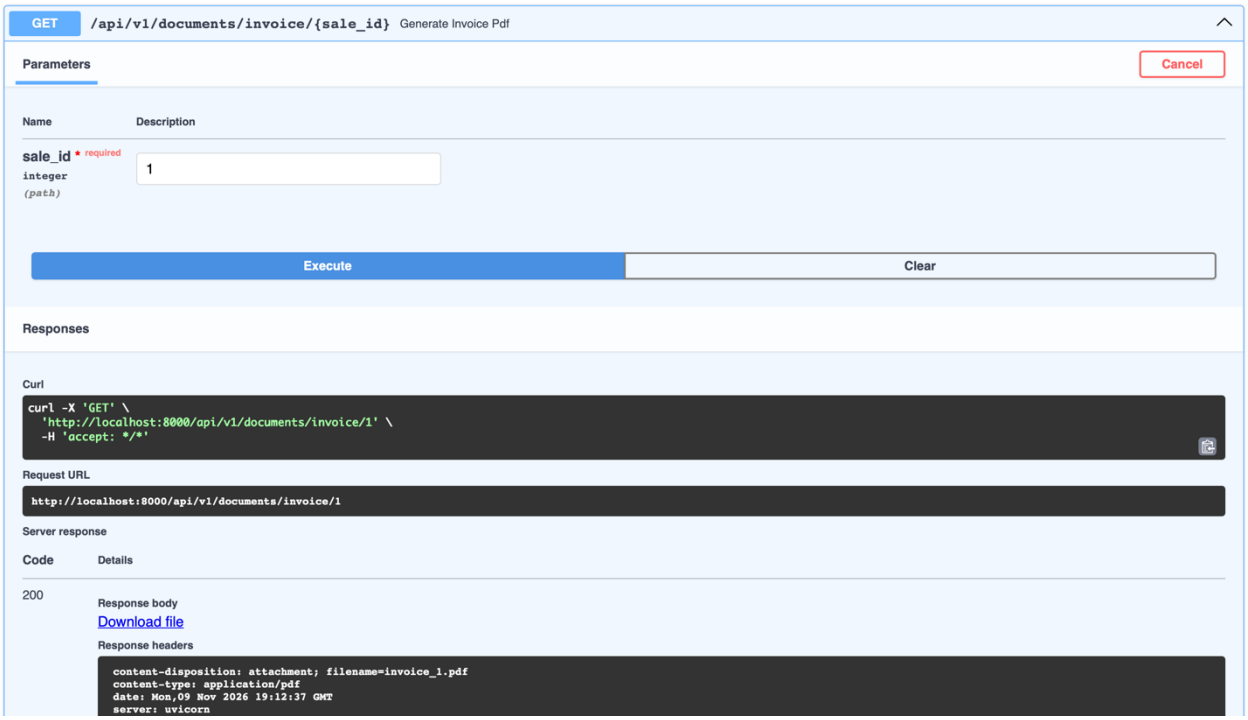


Рисунок 16 – Проверка скачивания PDF – накладной через API

На рисунке 17 представлена сама сгенерированная Товарная накладная в формате PDF с корректным отображением кириллицы и таблицей данных.

Товарная накладная

Номер чека: 1

Дата: 03.09.2026

Наименование	Кол-во	Цена	Сумма
Мука пшеничная	4	80.00	320.00
		ИТОГО:	**320.00**

Рисунок 17 – Сгенерированная Товарная накладная (PDF-файл) для чека № 1

Аналогично была проверена генерация Excel – файла. Для Excel – ведомости параметров вообще нет, так как она всегда выгружает текущий остаток по всему складу. На рисунке 18 представлен ответ сервера со ссылкой на скачивание.

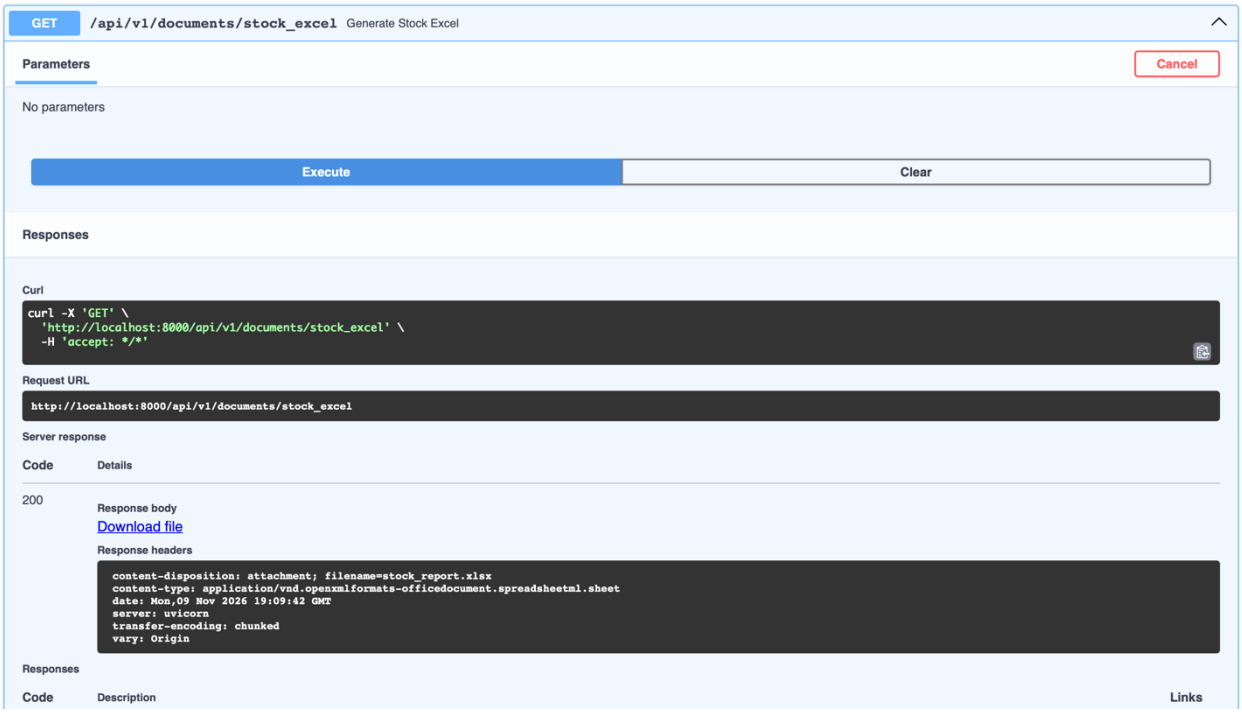


Рисунок 18 – Проверка скачивания Excel – файла через API

На рисунке 19 представлен сгенерированный отчет по остаткам в формате Excel.

ID	Название	Цена	Остаток	Мин. остаток
1	Молоко 3,2%	85	12	5
2	Кефир 1%	90	8	5
3	Сметана 20%	150	4	6
4	Творог 5%	120	15	10
5	Сыр Российский	320	3	4
6	Хлеб Бородинский	45	8	10
7	Батон нарезной	40	20	10
8	Булочка с маком	35	12	8
9	Мука пшеничная	80	25	10
10	Сахар-песок 1кг	75	30	15
11	Соль поваренная	25	40	20
12	Рис круглозерный	110	15	8
13	Гречка ядрица	105	18	10
14	Макароны спагетти	90	22	10
15	Масло подсолнечное	130	14	5
16	Куриное филе	290	5	6
17	Яйца куриные С1	110	20	15
18	Сосиски молочные	250	7	5
19	Вода питьевая 1,5л	50	50	20
20	Сок яблочный 1л	95	18	10
21	Чай черный	150	8	5
22	Кофе растворимый	350	6	4
23	Яблоки Гала	120	30	15
24	Бананы	95	25	20
25	Апельсины	140	10	15
26	Картофель	45	60	30
27	Морковь	40	35	20
28	Лук репчатый	35	40	20
29	Шоколад молочный	130	2	10
30	Печенье овсяное	85	12	5

Рисунок 19 – Сгенерированная ведомость остатков (Excel)

Выводы по практической работе.

В ходе выполнения практической работы были получены навыки агрегации данных средствами базы данных, визуализации данных с помощью библиотеки «Chart.js», а также формирования документов установленных форматов (PDF и Excel) на стороне сервера с корректным отображением кириллицы.

Исходный код клиентской части, расположенный в репозитории «sklad-magazina» на GitHub доступен по ссылке: https://github.com/mcnatusik-design/sklad-magazina.

Исходный код серверной части, расположенный в репозитории «sklad-backend» на GitHub доступен по ссылке: https://github.com/mcnatusik-design/sklad-backend.